**Merge Sort Time Complexity Analysis**

Let's analyze the time complexity of the two functions and then determine the overall time complexity of the merge sort algorithm.

**1. asc_merge_sort Function**

```
void asc_merge_sort(int A[],int left,int mid,int right){
   int n1=mid-left+1; // size_left → O(1)
   int n2=right-mid;   // size_right → O(1)

   int L[n1],R[n2];    // Create temp arrays → O(n1 + n2) space

   // Copy data to temp arrays → O(n1)
   for(int i=0;i<n1;i++){
      L[i]=A[left+i];
   }
   // Copy data to temp arrays → O(n2)
   for(int i=0;i<n2;i++){
      R[i]=A[mid+1+i];
   }

   int i,j,k;
   i=0;j=0;k=left;
```

```
    // Merge the temp arrays back → O(n1 + n2) = O(n)
    while(i<n1 && j<n2){
        if(L[i]<=R[j]){
            A[k]=L[i];
            i++;
        }else{
             A[k]=R[j];
             j++;
        }
        k++;
    }

    // Copy remaining elements → O(n1 - i) (worst case)
    while (i < n1) {
        A[k] = L[i];
        i++;
        k++;
    }

    // Copy remaining elements → O(n2 - j) (worst case)
    while (j < n2) {
        A[k] = R[j];
        j++;
        k++;
    }
}
```

**Time Complexity of asc_merge_sort:**

- All operations are linear relative to the size of the subarrays (n1 + n2 = n)
- Total time complexity: O(n)

**2. asc_merge_divide Function**

```
void asc_merge_divide(int A[],int left,int right){
   if(left>=right){         // Base case → O(1)
      return;
   }
   int mid = left + (right-left)/2;  // Find mid → O(1)

   // Recursively sort first half → T(n/2)
   asc_merge_divide(A,left,mid);

   // Recursively sort second half → T(n/2)
   asc_merge_divide(A,mid+1,right);

   // Merge the sorted halves → O(n)
   asc_merge_sort(A,left,mid,right);
}
```

**Time Complexity of asc_merge_divide:**

- This is the recursive divide step of merge sort
- At each level, it:
    1. Divides the problem into two subproblems of size n/2
    2. Performs O(n) work to merge the results

**Overall Time Complexity**

The merge sort algorithm follows the recurrence relation:
$T(n) = 2T(n/2) + O(n)$

This is a classic divide-and-conquer recurrence that can be solved using the Master Theorem or by analyzing the recursion tree:

1. **Recursion Tree Analysis:**
    - At each level i, there are 2^i subproblems, each of size n/(2^i)
    - The work done at each level is O(n) (from the merge step)
    - The tree has $\log_2 n$ levels (since we keep dividing by 2 until we reach size 1)
    - Total work = O(n) per level × $\log_2 n$ levels = O(n log n)
2. **Master Theorem:**
    - For $T(n) = aT(n/b) + f(n)$:
        - a = 2 (number of subproblems)
        - b = 2 (division factor)
        - f(n) = O(n)
    - Case 2 applies since $f(n) = \Theta(n^{\log_a b}) = \Theta(n^{\log_2 2}) = \Theta(n)$
    - Solution is T(n) = O(n log n)

**Best Case:** O(n log n)
**Average Case:** O(n log n)
**Worst Case:** O(n log n)

Merge sort always divides the array into two halves and takes linear time to merge them, resulting in consistent O(n log n) performance in all cases.